

PA-03 Traffic Generation

Responding to ARP Requests & ICMP Echo Requests

Relevant Global Variables in mypcap.c

```
FILE      *pcapInput  = NULL ;
FILE      *pcapOutput = NULL ;

arpmap_t   myARPmap[ MAXARPMAP ] ; // List of my IPs, their MACs
int        mapSize    = 0;          // Number of mapping pairs read into above array
unsigned   countMine  = 0;
```

Where, this new type `arpmap_t` is to be defined in your `mypcap.h`

```
/* IP-to-MAC mapping database used by ARP */
typedef struct
{
    uint32_t   ip ;
    uint8_t    mac[ ETHERNETLEN ] ;
} arpmap_t ;
```

Main Task:

- Your main() in p3.c must accept three command-line arguments:
 1. input PCAP file name: some previously-captured traffic
 2. output PCAP file name: a new PCAP file you will create whose global header that is to be identical to that of the input PCAP file
 3. a text file containing an undefined list of ARP pairs (IP address, MAC address). These are the MAC and IP addresses of your machine. I attached a sample [arp.dat](#) for this purpose.
- Read the list of ARP mappings: (my IP , my MAC) from the text file into the global array `myARPmap[]`, then print them to the standard output device.
- Iterate through all captured packets in the input PCAP file, printing their destination MAC addresses and whether they are your MAC addresses (see sample run). You shall respond only to ARP request and Echo request packets targeting your machine (as determined by the list of ARP mappings you read above). You must not respond to any other packets (neither should you duplicate them to the output PCAP).
- For each ARP request, or proper ICMP Echo request targeting your machine:

1. Duplicate the packet to the output PCAP file as is. Keep a count of these packets
 2. Prepare the appropriate response packet, and export it to the same output PCAP file.
- Your output PCAP file must be properly recognized by Wireshark with IP header checksum and ICMP checksum correctly computed by your program.

Question 1: How do you decide that an Ethernet frame is targeting your machine?

Question 2: Next, how do you determine that it's an ARP request asking about your machine's MAC address?

Question 3: How do you determine it's a proper ICMP echo request targeting your machine?

New functions to add to mypcap.c:

```
void processRequestPacket( packetHdr_t *pktHdr, uint8_t ethFrame[] );
```

This is where most of the effort is done. You have just read a packet from the input PCAP file where `pktHdr` points at its packet header, and its captured Ethernet frame is in `ethFrame[]`. You shall print the destination MAC of this packet, and whether that address is one of yours. If this packet destination MAC does NOT target your machine, simply return from here. Do NOT even copy this packet to the output PCAP file.

Otherwise: Respond only to an incoming ARP Request or proper ICMP Echo Request targeting your machine as follows:

- Copy the request packet (header + Ethernet frame) as is to the output PCAP file,
- Followed by your reply packet (header + Ethernet frame). This reply packet must have a capture time that is **30 microseconds** after that of the request packet.
- Moreover, for your ICMP Echo reply packets:
 - the starting IP Identification must be 1000 (in decimal), then incremented by 1 for successive IP datagrams you generate to the output.
 - **The IP flags should indicate "Do Not Fragment"**

```
int writePCAPhdr( char *fname , pcap_hdr_t *p ) ;
```

- Open the output PCAP file '`fname`' and write its global header from pre-filled info in buffer '`p`'
- Returns: 0 on success, -1 on failure

```
int readARPmap( char *arpDB ) ;
```

- Read IP-to-MAC mappings from file '**arpDB**' into the global array:

```
arpmap_t    myARPmap[ MAXARPMAP ] ;
```

Returns: the actual number of mappings read from the file (setting the global 'mapSize' to that same number), or -1 on failure

```
uint16_t inet_checksum( void * data , uint16_t lenBytes );
```

- Compute and return the Internet Checksum using One-Complement Arithmetic on an array of **bytes** pointed to by '**data**', which has a total of '**lenBytes**' bytes (may be an even or odd value).

```
bool myIP( IPv4addr someIP , uint8_t **ptr );
```

- Check if '**someIP**' is one of mine. If not mine, and if **ptr** is not NULL, then set ***ptr** to NULL. If '**someIP**' is mine, and if **ptr** is not NULL, then set ***ptr** to point at the corresponding MAC value inside the global '**myARPmap[]**' array. Note that no copying of the actual bytes of the MAC address is done.

```
bool myMAC( uint8_t someMAC[] );
```

- Check if 'someMAC' is one of mine. Note that a MAC broadcast address must also be treated as mine.

Feel free to add any additional utility functions to mypcap.c.

Things to Consider:

- When preparing an ICMP echo reply, you will have to modify several fields of the outgoing packet's IP header compared to that of the incoming request. This requires the re-calculation of the header's checksum for the outgoing Echo Reply. Study how to calculate the Internet Checksum in your textbook: **Appendix D, Figure D.2**
- **Question 4:** Which fields of the outgoing packet's IP header need to be modified?
- **Question 5:** Do you need to recalculate the checksum of the IP header? of the ICMP message? or both?